

ESE589 Decision Tree Project

| Lichen Yang, 116472922

This project implements the **Decision Tree Algorithm** using Python. The implementation is validated with a small illustrative example and benchmarked on the real-world Census Income dataset from UCI Machine Learning Repository.

1 Dataset and Preprocessing

Choice of dataset

The benchmark dataset chosen for the project is the Census Income dataset (also known as the Adult dataset) from UCI Machine Learning Repository. This dataset is a classic benchmark for binary classification tasks and contains demographic information to predict whether an individual's annual income exceeds \$50,000.

The census dataset is a good selection for decision tree algorithm because it typically contains a mix of categorical and numerical features, making it well-suited for decision tree algorithms, which can handle both types of data effectively. Additionally, the dataset often includes a variety of attributes (e.g., age, education, income) that can provide meaningful splits, leading to interpretable decision rules. Some details about this dataset is listed here:

- **Size:**
 - Training set: ~32,000 samples
 - Test set: ~16,000 samples
- **Features:** 14 attributes including:
 - **Continuous features:** age, fnlwgt, education-num, capital-gain, capital-loss, hours-per-week
 - **Categorical features:** workclass, education, marital-status, occupation, relationship, race, sex, native-country
- **Target Variable:** Income (binary: $\leq 50K$ or $> 50K$)

- **Class Distribution:** Imbalanced (~75% $\leq 50K$, ~25% $> 50K$)

Example contents of the dataset

39, State-gov, 77516, Bachelors, 13, Never-married, Adm-clerical, Not-in-family, White, Male, 2174, 0, 40, United-States, $\leq 50K$

50, Self-emp-not-inc, 83311, Bachelors, 13, Married-civ-spouse, Exec-managerial, Husband, White, Male, 0, 0, 13, United-States, $\leq 50K$

38, Private, 215646, HS-grad, 9, Divorced, Handlers-cleaners, Not-in-family, White, Male, 0, 0, 40, United-States, $\leq 50K$

Preprocessing

The data preprocessing involves the following steps:

- **Data cleaning:** including removing rows containing missing values, stripping trailing '.' characters, and merging train and test sets temporarily for consistent encoding.
- **Categorical Encoding:** Applied Label Encoding to all categorical features
- **Target Encoding:** Converted binary income labels ($\leq 50K$, $> 50K$) to integers (0, 1)

After these steps, there are 30162 samples in the training set and 15060 samples in the test set. We kept all of the 14 features for training, and used the original train/test split of the dataset.

2 Implementation Details

My implementation of the decision tree algorithm can be found in `decision_tree.py`.

The implementation follows an object-oriented design with the `DecisionTreeClassifier` class as the main component.

Node Class: Represents individual nodes in the tree. It stores feature index, threshold value, child pointers, and leaf values, and supports both internal decision nodes and leaf prediction nodes.

DecisionTreeClassifier Class: this is the main classifier with configurable parameters:

- `criterion`: Choice of splitting metric ('information_gain', 'gain_ratio', 'gini_index')

- `max_depth` : Maximum tree depth to prevent overfitting
- `min_samples_split` : Minimum samples required to split a node
- `min_samples_leaf` : Minimum samples required at leaf nodes

Building the Decision Tree

- **Initialize the Tree:** Start with an empty root node with unknown feature index and threshold value.
- **Recursive Tree Building:** for most nodes that don't meet the stopping criteria, the best split is found following these steps:
 - Find best feature and threshold using selected criterion
 - Split data into left ($\leq$ threshold) and right ($>$ threshold) subsets
 - Check `min_samples_leaf` constraint
 - This process is executed recursively to build left and right subtrees, until the entire decision tree is complete.

How to Run My Code

First, make sure you have the right python packages by running:

```
pip install -r requirements.txt
```

Then directly run `validation_examples.py` to validate the correctness of my implementation with a simple synthetic test case.

Run `benchmark.py` for comprehensive experiments on the Census Income dataset. This will first attempt to download and preprocess the Census Income dataset, then perform a series of experiments and print the results.

Run `comparison.py` to experiment the three splitting criteria on various datasets.

The implementation of decision tree is written in `decision_tree.py` and it is not meant to be run directly.

3 Example for Validation

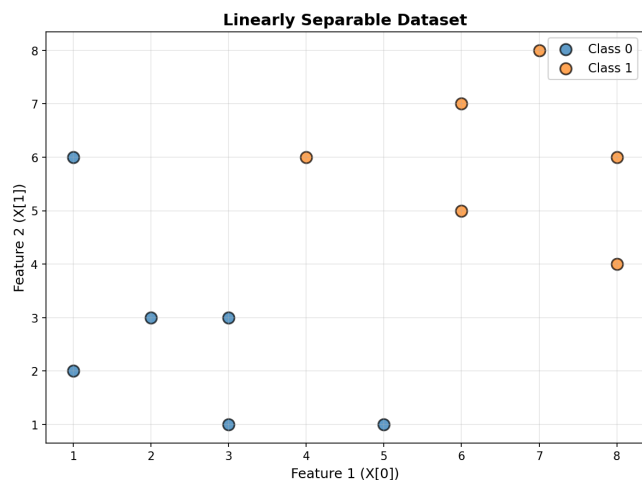
To ensure the correctness and robustness of my decision tree algorithm implementation, I used small, illustrative example to validate the algorithm. This can be found in `validation_examples.py` .

The used test example and results are listed below.

The Simple Linearly Separable Dataset Used

I constructed a simple linearly separable dataset for binary classification. This means that the dataset has two clusters in 2D space that each belongs to one category, and they can be perfectly separated by a straight line. The dataset is shown below, as well as a visualization image to better illustrate the content in this dataset.

```
X = np.array([
    [1, 2],
    [2, 3],
    [3, 1],
    [5, 1],
    [3, 3],
    [1, 6],
    [6, 5],
    [7, 8],
    [8, 6],
    [4, 6],
    [6, 7],
    [8, 4],
])
y = np.array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])
```



Test 1: Examine the prediction of decision tree using different criteria

When I run the decision tree algorithm on this dataset using the three different criteria (information_gain, gain_ratio, gini_index), they all reach the same decision tree structure:

```
Root: if X[0] <= 3.0000:
    |— True: Predict class 0
    |— False:
        |— if X[1] <= 1.0000:
            |— True: Predict class 0
```

```
else:  
    └ False: Predict class 1
```

This is reasonable for the given training samples.

Test 2: Compare the result with the sklearn library

I compared our decision tree algorithm with the one implemented in the `sklearn` library. Using the same simple illustrative dataset introduced above and the same criteria (`gini_index`), the output of the two algorithms matches perfectly.

Therefore we can see that with the simple linear separable dataset, all criteria can achieve 100% accuracy. The decision tree structure is simple and reasonable, and comparison with `sklearn` library shows similar prediction result. Therefore we can verify the correctness of the decision tree implementation.

4 Experiment on the three metrics

I tested the decision tree algorithm using different synthetic dataset and 3 different metrics (`information_gain`, `gain_ratio`, `gini_index`). The results, including test accuracy and time performance, are shown below, followed by a brief summary.

Experiment 1: Linearly Separable Data

Dataset Info:

Samples: 1000 | Features: 10 | Class distribution: [500, 500]

Criterion	Train Acc	Test Acc	Time (s)
Information Gain	1.0000	0.9633	0.5331
Gain Ratio	0.9986	0.9567	1.3159
Gini Index	1.0000	0.9700	0.6005

Best: Gini Index — *0.9700 test accuracy*

Experiment 2: Complex Decision Boundary

Dataset Info:

Samples: 1000 | Features: 10 | Class distribution: [498, 502]

Criterion	Train Acc	Test Acc	Time (s)
Information Gain	0.9814	0.7833	0.8570
Gain Ratio	0.6229	0.6167	1.7239
Gini Index	0.9700	0.7700	1.0196

Best: Information Gain — *0.7833 test accuracy*

Experiment 3: Imbalanced Classes

Dataset Info:

Samples: 1000 | Features: 10 | Class distribution: [896, 104]

Criterion	Train Acc	Test Acc	Time (s)
Information Gain	1.0000	0.9400	0.6472
Gain Ratio	0.9457	0.9167	1.8806
Gini Index	1.0000	0.9467	1.3587

Best: Gini Index — *0.9467 test accuracy*

Experiment 4: High Dimensional Data

Dataset Info:

Samples: 1000 | Features: 50 | Class distribution: [500, 500]

Criterion	Train Acc	Test Acc	Time (s)
Information Gain	0.9714	0.7167	3.8745
Gain Ratio	0.6371	0.6133	8.7532
Gini Index	0.9871	0.6600	4.8450

Best: Information Gain — *0.7167 test accuracy*

Experiment 5: Non-linear Decision Boundary (Moons)

Dataset Info:

Samples: 1000 | Features: 2 | Class distribution: [500, 500]

Criterion	Train Acc	Test Acc	Time (s)
Information Gain	0.9786	0.9133	0.1925
Gain Ratio	0.9343	0.8700	0.2129

Criterion	Train Acc	Test Acc	Time (s)
Gini Index	0.9829	0.8967	0.2936

Best: Information Gain — *0.9133 test accuracy*

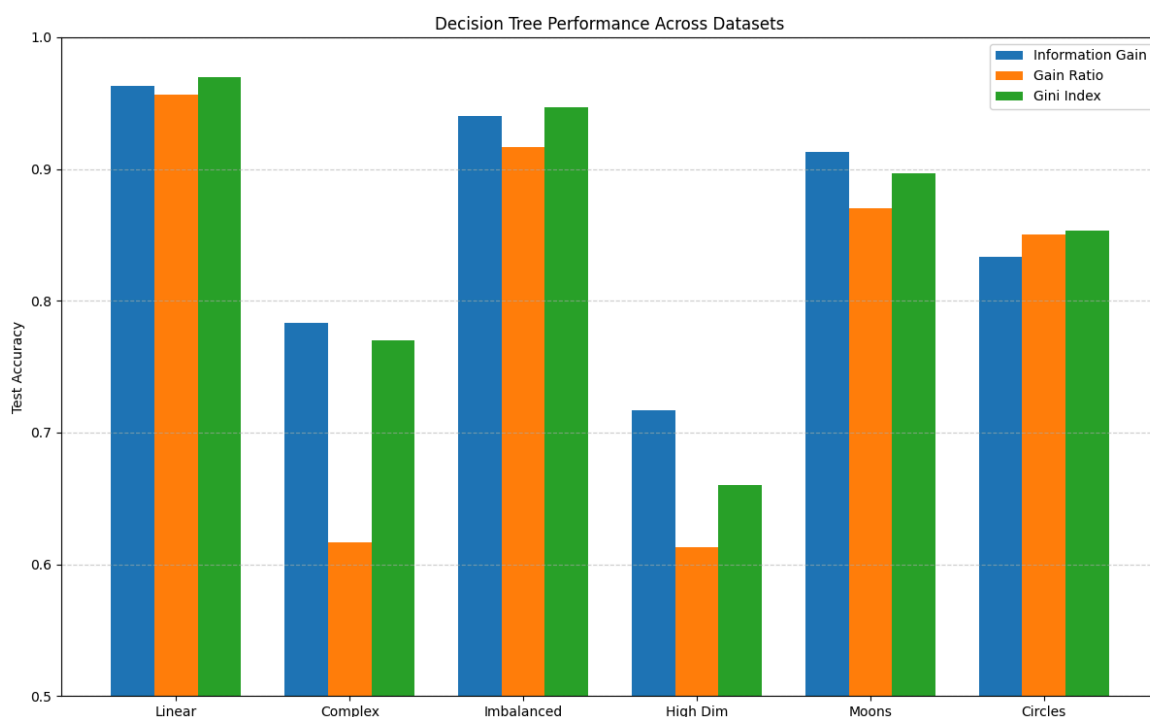
Experiment 6: Very Non-linear Boundary (Circles)

Dataset Info:

Samples: 1000 | Features: 2 | Class distribution: [500, 500]

Criterion	Train Acc	Test Acc	Time (s)
Information Gain	0.9529	0.8333	0.2447
Gain Ratio	0.9057	0.8500	0.3374
Gini Index	0.9557	0.8533	0.2355

Best: Gini Index — *0.8533 test accuracy*



Observations

From the above results we can see that:

1. All three criteria perform similarly on most datasets.
2. Gain Ratio tends to be slower due to its extra normalization step.

3. Gini Index and Information Gain usually provide the best trade-off between accuracy and runtime.
 4. The optimal criterion depends on data distribution and dimensionality.
-

5 Benchmark Result Discussion

Three experiment are performed on the **Adult Census Income Dataset**. The primary goal is to evaluate the performance of different splitting criteria (`information_gain` , `gain_ratio` , `gini_index`) and further investigate the impact of tree depth and training set size on classification performance.

Performance Comparison of Splitting Criteria

First, we compare the performance of 3 different metrics (`information_gain`, `gain_ratio`, `gini_index`). For this experiment, `max_depth` is set to be 10 and 5000 samples randomly selected from the training set are used as training data.

Splitting Criterion	Training Accuracy	Test Accuracy	Training Time (s)
<code>information_gain</code>	0.8546	0.8458	1.43
<code>gain_ratio</code>	0.7876	0.7969	1.14
<code>gini_index</code>	0.8626	0.8402	1.58

- **Best Splitting Criterion:** `information_gain` , achieving the highest test accuracy of **0.8458**.
- `gain_ratio` had the lowest test accuracy but the shortest training time.
- `gini_index` achieved the highest training accuracy but slightly lower test accuracy compared to `information_gain` .

Effect of Tree Depth on Performance

For this experiment, we use `information_gain` as the splitting criteria and still use 5000 randomly selected samples as training data.

Max Depth	Training Accuracy	Test Accuracy	Training Time (s)
5	0.8508	0.8378	1.24
10	0.8466	0.8373	1.54
15	0.8618	0.8372	1.60

Max Depth	Training Accuracy	Test Accuracy	Training Time (s)
20	0.8618	0.8387	1.45

- Test accuracy reached near-optimal levels at a depth of **5**, with further increases in depth providing limited improvement.
- Training accuracy increased with depth, but test accuracy stays almost the same, indicating that deeper trees may lead to overfitting, and slightly higher training cost.

Effect of Training Set Size on Performance

For this experiment, we use `information_gain` and `max_dept = 10`. Different training set size is tested to compare their performance.

Training Sample Size	Training Accuracy	Test Accuracy	Training Time (s)
1000	0.8260	0.8288	0.17
2500	0.8728	0.8313	0.55
5000	0.8526	0.8337	1.59
10000	0.8534	0.8438	4.75

- Test accuracy improved significantly as the training sample size increased, particularly from **1000** to **10000** samples, where test accuracy rose from **0.8288** to **0.8438**.
- Training time increased linearly with sample size, requiring a trade-off between training time and performance.

6 Summary

This project successfully implemented and validated the decision tree algorithm. Through systematic testing on both synthetic and real-world datasets, the implementation demonstrated correct behavior on a synthetic simple dataset for illustration purposes. The benchmark results on the Census Income dataset revealed key performance characteristics:

1. **Best Splitting Criterion:** `information_gain` performed the best on the test set most of the time.
2. **Tree Depth:** Tree depth had a limited impact on performance. It is recommended to limit the depth to **5-10** to avoid overfitting.

3. **Training Sample Size:** Increasing the training sample size significantly improved test accuracy, but training time also increased. The choice of sample size should balance performance and computational cost.